REQUIREMENTS

Mei Nagappan



UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

Guideline 1: Identify Stakeholders First

- Before generating any requirements, ask the model to identify all stakeholder groups. Then, for each group, run a separate prompt instructing the model to adopt that group's perspective. This two-step sequence catches non-user stakeholders that are silently omitted in direct generation.

Reasoning

- The Persona Pattern instructs the model to adopt a specific role before responding; prompt patterns can be chained so the output of one step becomes input to the next.

- The root cause of missing non-user requirements is that practitioners working in unfamiliar domains do not know to look for those stakeholders in the first place.

- Automated tools reproduce the same stakeholder blind spots as practitioners when given no explicit instruction to look beyond end users — the identification step must be a separate, explicit prompt.

Key Practices: Step 1 — Stakeholder Identification

- Prompt: "List all stakeholder groups who have a stake in this system. Include not just end users but also anyone who operates, deploys, maintains, supports, or audits it. For each group, write one sentence describing their primary concern."
- Do not skip this step even if you think you already know the stakeholders.
- Treat the model's list as a starting point. Add missing roles and remove those that do not apply to your project.

Key Practices: Step 2 — Per-Stakeholder Requirements

- For each group, run a separate prompt: "Now adopt the perspective of a [role]. Generate at least three requirements not already covered by existing requirements."
- Do not merge multiple personas into one prompt as the model will collapse them and the least-visible groups will still be lost.
- Risk: The model's stakeholder list skews toward common system types. Always cross-check against domain standards or regulations before proceeding to Step 2.

Guideline 2: Flipped Interaction Pattern

- Instruct the model to drive the conversation by asking you targeted questions one at a time until it has enough information to produce a complete requirements document, instead of telling the model what you want up front.

Reasoning

- Flipped Interaction Pattern: model asks questions to surface assumptions, not hallucinate them from a thin description.
- LLM requirements from thin prompts are abstract and infeasible; structured questioning forces confirmation before filling gaps.
- Iterative prompting grounds RE outputs in confirmed stakeholder intent, not statistical plausibility.

Key Practices: Elicitation

- Open with: "Ask me questions one at a time about my software system until you have enough information to write a complete SRS. Start with the most important question."
- Do not volunteer a long description up front; let the model discover gaps through its questions.
- After each answer, confirm the model's understanding before it moves to the next question.

Key Practices: Review

- After the session, ask: "Based on our conversation, list any assumptions you made that I have not explicitly confirmed."
- Check that every generated requirement traces back to at least one answer you gave during the session.
- Reject any requirement containing "appropriate", "reasonable", or "user-friendly" without a measurable criterion attached.

Guideline 3: Cognitive Verifier Pattern

- When a stakeholder provides a vague or overloaded requirement, prompt the model to decompose it into specific sub-questions before generating any acceptance criteria. Answer those sub-questions yourself, then ask the model to write the final requirement.

Reasoning

- Cognitive Verifier Pattern: subdivide questions into sub-questions first, forcing ambiguities to the surface before they reach acceptance criteria.
- Single-shot prompts produce underspecified output because the model fills gaps with statistically common answers, not correct domain answers.

Key Practices

- Open with: "Before generating any requirements for the following statement, list the sub-questions you need answered to write a complete, unambiguous requirement. Do not answer the questions yourself."
- Answer every sub-question yourself in a follow-up message, then ask the model to generate the final requirements, explicitly referencing your answers.
- If the model introduces a term you did not define, ask it to flag the assumption and propose two alternatives for you to choose.

References (1 of 2)

- [1] White et al. (2023). ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arXiv:2303.07839.
- [2] Ronanki et al. (2023). Investigating ChatGPT's potential to assist in requirements elicitation processes. Euromicro SEAA, pp. 354-361. IEEE.
- [3] Ronanki et al. (2024). Requirements engineering using generative AI: Prompts and prompting patterns. In Generative AI for Effective Software Development. Springer. arXiv:2311.03832.
- [4] Quattrocchi et al. (2025). Can LLMs generate user stories and assess their quality? arXiv:2507.15157.
- [5] Almonte et al. (2025). Automated non-functional requirements generation in software engineering with large language models. arXiv:2503.15248.
- [6] Pasquale et al. (2025). Exploring the use of LLMs for requirements specification in an IT consulting company. arXiv:2507.19113.

References (2 of 2)

- [7] Zadenoori et al. (2024). Large language models for requirements engineering: A systematic literature review. arXiv:2509.11446.
- [8] Hemmat et al. (2025). Research directions for using LLM in software requirement engineering: A systematic review. Frontiers in Computer Science. DOI: 10.3389/fcomp.2025.1519437
- [9] Zhu et al. (2025). ReqInOne: A large language model-based agent for software requirements specification generation. arXiv:2508.09648.
- [10] ISO/IEC/IEEE 29148:2018. Systems and Software Engineering -- Life Cycle Processes -- Requirements Engineering. DOI: 10.1109/IEEESTD.2018.8559686
- [11] Khlood et al. (2023). Requirements elicitation and modelling of AI systems. ENASE, pp. 15-26. DOI: 10.5220/0011863200003464
- [12] Habibullah et al. (2023). Non-functional requirements for machine learning. Requirements Engineering, 28(2), 283-316. DOI: 10.1007/s00766-022-00379-z